

Week 10: Packaging & Launch

Objectives

- Prepare the final build
- Finish pdf for submission

Task Breakdown

- Package the game
- Finalize code comments, and project report
- Clean commit history, final push to GitHub

Deliverables

- Final packaged, runnable game
- Complete README.md
- Final project report
- Final GitHub repository

Tools & Resources

- Python 3.11+
- Tkinter (built-in)
- VS Code/PyCharm
- Git

Emoji Hangman

A polished emoji-based Hangman game built with **Python 3.11+** and **Tkinter**, developed for *CSE 2216 — Software Development I*.

Features

-  **10-word campaign** across 3 levels (Easy → Medium → Hard)
-  **Emoji clues** for every word
-  **10-second timer** per word — with **Pause** support
-  **5 lives** per game
-  **Hint system** (reveals one letter, costs 10 points)
-  **6 achievements** — First Win, 5 Streak, Hint Master, Speed Solver, Survivor, Champion
-  **Player rating** based on achievement progress
-  **Persistent leaderboard** (top 10 with medals) — saved to scores.json
-  **Clear leaderboard** with confirmation dialog
-  **Keyboard shortcuts**: Enter = guess, Space = pause, Esc = menu
-  **Graceful handling** of missing or corrupted save files

Project Structure



PROJECT_REPORT.md

Final Project Report — Emoji Hangman

****Course:**** CSE 2216 — Software Development I

****Institution:**** Northern University of Business & Technology, Khulna

****Team:**** <Your Name>

****Submission date:**** <DD/MM/YYYY>

1. Introduction

Emoji Hangman is a modern re-imagining of the classic word-guessing game. Instead of revealing a hangman's gallows with each wrong guess, players are given an ****emoji clue**** and must decode the underlying word. The project was completed over ten weeks, following an incremental delivery model.

2. Problem Statement

Traditional Hangman is easy to code but visually dull and offers no persistence. Our goal was to build a ****visually polished****, ****feature-rich**** desktop game that:

1. Uses emojis as clues (modern, appealing to all ages),
2. Persists scores between sessions,
3. Rewards skill with achievements and player ratings,
4. Handles edge cases (missing files, corrupted data, empty input) gracefully.

3. Scope

In scope

- Single-player desktop game (Windows/macOS/Linux with Tk)
- 3-level campaign (10 words total)
- Achievements, player rating, leaderboard, hint system, pause, timer
- Save/load system using JSON

Out of scope

- Multiplayer, networking, mobile, web

4. Methodology

Agile-style weekly sprints:

Week	Focus
-----	-----
1–3	Core game loop, word loading, guessing logic
4–5	Levels, timer, lives, hint system

6-7	Achievements, player rating, pause button
8	Leaderboard & save system
9	Testing & bug fixing
10	Packaging & documentation

Each week ended with a runnable build and a short retrospective.

5. Implementation

5.1 Architecture

Single-class UI (`HangmanApp`) + pure I/O module (`leaderboard.py`) + JSON data (`emoji\_data.json`, `scores.json`).

5.2 Key algorithms

- **Word selection**: filter out used words, pick uniformly at random; refill pool when exhausted.
- **Timer**: `after(1000, ...)` loop; pause cancels + restarts.
- **Achievements**: checked on each solved word.
- **Persistence**: atomic write + corruption backup.

5.3 UI/UX

Dark slate theme, vivid accents, emoji-centred puzzle cards, keyboard shortcuts (`Enter`, `Space`, `Esc`). Every screen fits in a single 780×680 window.

6. Testing Summary

30 test cases executed, 15 bugs logged. **14 fixed**, 1 marked WONTFIX by design. See `BUGS.md` and `TESTING\_REPORT.md`.

7. Results

The final build:

- Launches in < 1 s.
- Runs on Python 3.11+ with **zero external dependencies**.
- Handles all edge cases without crashing.
- Passed peer testing with positive feedback.

8. Challenges & Learnings

| Challenge | Learning |

|-----|-----|

Timer crashing after screen switch	Always cancel `after()` jobs on teardown
Tkinter emoji rendering	Requires `Segoe UI Emoji` on Windows
Corrupted scores.json	Defensive `try/except` + backup strategy
Duplicate achievements per round	Track two separate sets
Layout overflow	Use grid + `columnconfigure(weight=1)`

9. Future Work

- Sound effects
- Difficulty selector
- Daily challenge
- CSV export
- Web version with PyScript

10. Conclusion

Emoji Hangman demonstrates that a small, well-scoped Python project can deliver a **polished, fun, and robust** experience. All Week 8–10 deliverables were met: a working save/load system, a bug-free build, complete documentation, and a final packaged project.

Appendix A — Files Delivered

- `emoji\_hangman.py`
- `leaderboard.py`
- `emoji\_data.json`
- `scores.json` (auto)
- `README.md`
- `DESIGN.md`
- `BUGS.md`
- `TESTING\_REPORT.md`
- `PROJECT\_REPORT.md`
- `requirements.txt`

Appendix B — Screenshots

(Insert screenshots of menu, game, victory, leaderboard, achievements.)